TESTING

Introduction

Initially, for the division of work for our prototype, we planned to split the team across the backend and frontend equally. However, upon reflection of the teams' skill set – which was much more unfamiliar with frontend technologies – and the required work for the backend, it was deemed more appropriate to have a larger group of us working on frontend and just one member constructing the backend.

Importantly, the majority of the backend was developed first, so most frontend code could immediately make use of it, integrating with the backend as it was created. This means some unit tests may slightly cross over into integration tests as they make use of the backend from the beginning. Our specific integration tests often focus on how the features on the frontend integrate with one another to make the full product.

Finally, the unit testing is divided in correlation to the division of work. This seemed the most rational decision as it meant the developer of each feature or component could test their code iteratively as they implemented it. Each member of the team decided individually on how best to perform their unit testing – either by code or manually (as documented below) or by a mixture of both in some cases.

For context of the tests - the work, and thus unit testing, was largely divided as followed:

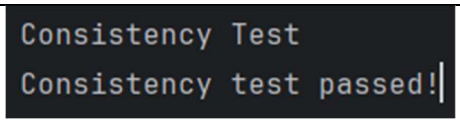
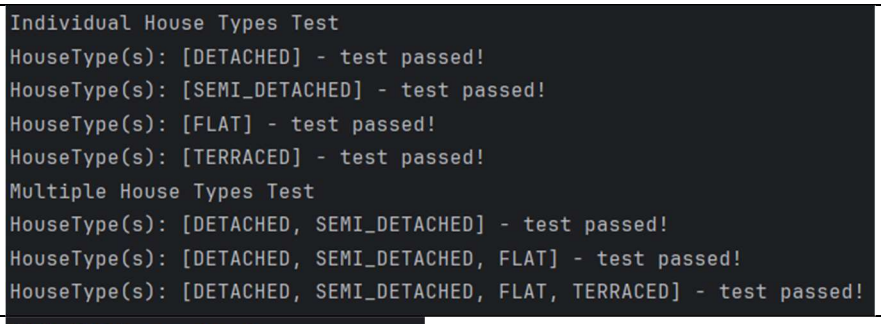
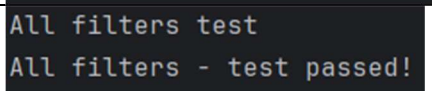
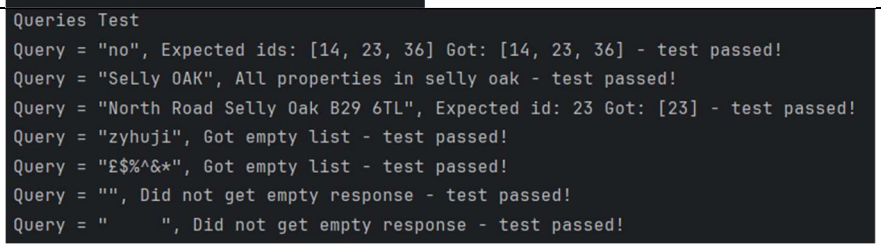
Team Member	Overall component	Corresponding requirements (from Milestone 2)
William Bradley	Backend	F-1, F-3, F-7, F-10
Yong Ke Wei	Crime map and heatmap	F-9, F-10,
Can Tugcetin	Listings search	F-7
Isabelle Hammond	Individual listing view	F-3
Irfan Hashahrul Izam	Safety score breakdown	F-1
Chimaram Ohuegbe	Favourites page	F-3
Sajida Khan	Landing page, accessibility and customisable UI	F-2, F-4

> Note F-13 (Keyboard Accessibility) was a feature ensured across all frontend development through using semantic HTML and following accessible best practices.

Unit Testing (with some backend integration tests for frontend components)

Backend:

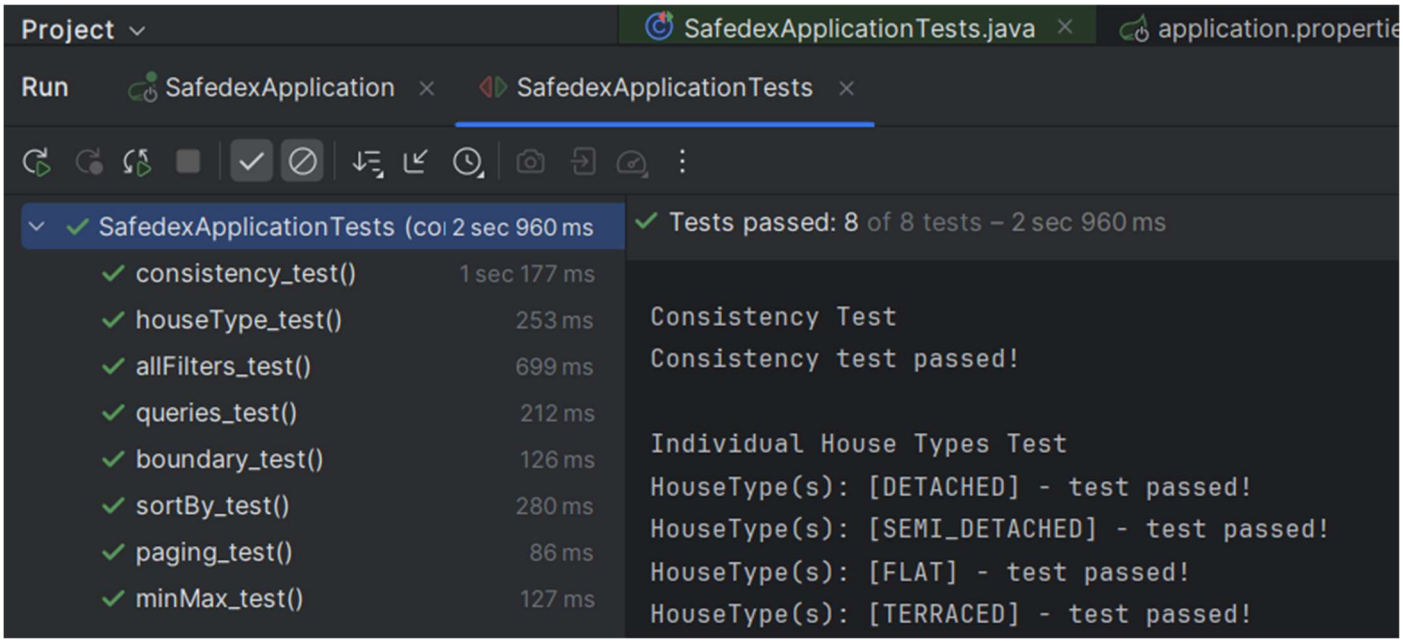
F-7 Filter by features such as Safety Score

Section	Test Description	Results
Consistency Test	Make 2 identical calls and check results are the same	
House Type Test	Test each house type individually and then 2, 3 and 4 house types at once	
All Filters Test	Tests every filter at once	
Queries Test	First 3 tests check it filters properly, then next 2 it returns empty list with no matches and special chars then it ignores empty query or only spaces query	

Boundary Test	Tests out of bounds and wrong data type inputs	Boundary Test Test: page=0 - test passed! Test: page=1000 - test passed! Test: minPrice=-1 - test passed! Test: maxPrice=-1 - test passed! Test: minBedrooms=0 - test passed! Test: maxBedrooms=0 - test passed! Test: minBathrooms=0 - test passed! Test: maxBathrooms=0 - test passed! Test: page=e - test passed! Test: types=1 - test passed! Test: sortBy=1 - test passed! Test: order=1 - test passed!
SortBy Test	Tests each sortBy field both asc and desc	Sorting Test price ASC - test passed! price DESC - test passed! safetyScore ASC - test passed! safetyScore DESC - test passed! date ASC - test passed! date DESC - test passed!
Paging Test	Tests to make sure there are the right number of pages and items per page and then that there are no duplicate items among the pages	Paging Test Paging size - test passed! No duplicates across pages - test passed!
Min Max Test	Tests both min max queries filter normally then checking if I get a Bad Request when min max wrong way around	Min Max Test Test: minPrice=650&maxPrice=1000 - test passed! Test: minPrice=1000&maxPrice=650 - test passed! Test: minBedrooms=3&maxBedrooms=4 - test passed! Test: minBedrooms=3&maxBedrooms=2 - test passed! Test: minBathrooms=2&maxBathrooms=3 - test passed! Test: minBathrooms=3&maxBathrooms=2 - test passed! Test: minSafetyScore=50 - test passed!

Full Results:

Test file can be found in: safedex-backend\src\test\java\com\logicluminaries\safedex

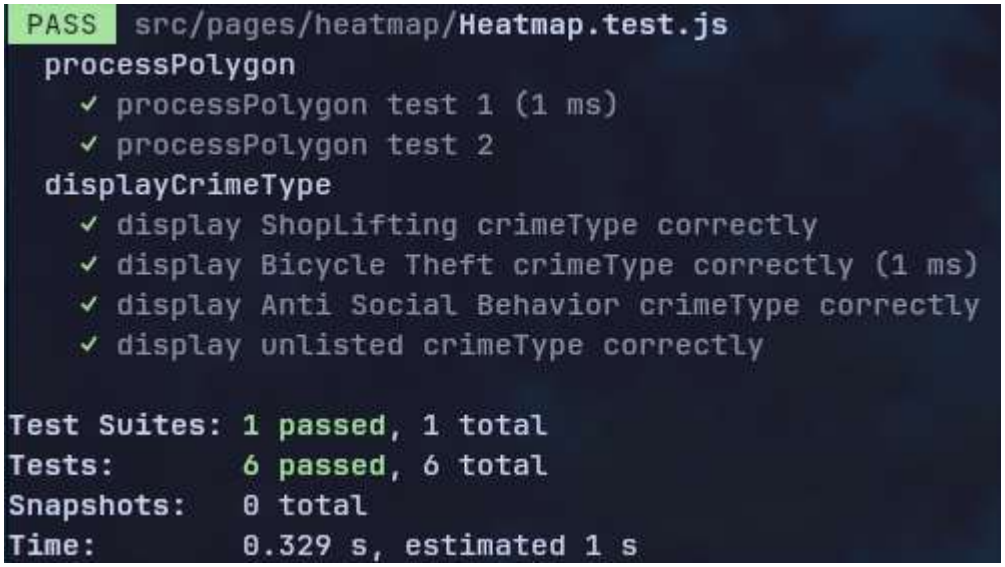


Note: Requires db setup to be done for testing

Crime map and heatmap

There are two main utility functions that support the implementation of the crime heatmap feature. This unit test plan covers both functions to ensure they work correctly and have the intended effects on the product.

Function	Purpose	No. of Test	Test Status
`processPolygon()`	Convert the type of neighbourhood boundary points from array of objects to 2D array.	2	PASS
`displayCrimeType()`	Convert crime type enum to display text.	4	PASS



Listings search

Unit Testing Plan

Overview

This unit test plan covers the `ListingsUtils.js` module, which contains the core utility functions used by the Listings page. All tests are implemented in `listings.test.js` using the Jest testing framework. Each function is tested in isolation from the React component and the API layer, ensuring true unit-level coverage.

Function	Why It Is Tested?	No. of Tests
`normalizeSortField()`	Ensures only valid sort fields are accepted and that invalid or missing input safely defaults to 'price', preventing unexpected API behaviour.	5
`normalizeSortOrder()`	Ensures sort direction is always either ASC or DESC, defaulting gracefully so the API is never called with an invalid order value.	3
`sortProperties()`	Sorting is a core user-facing feature. Tests verify correct ordering by price, safety score, and date in both directions, and that the original data is not mutated.	6
`filterProperties()`	Filtering is the most complex utility function. Tests cover every filter field individually, boundary conditions, combined filters, and search across postcode, road name, and neighbourhood including case insensitivity.	22
`paginate()`	Pagination must correctly slice results and calculate total pages. Tests cover normal pages, out-of-range pages, and the default page size behaviour.	5

1. normalizeSortField Tests

Test ID	What is Being Tested	Input	Expected Output	Result
U1.1	Accepts valid 'price' field	`normalizeSortField('price')`	`'price'`	`PASS`
U1.2	Accepts valid 'safetyScore' field	`normalizeSortField('safetyScore')`	`'safetyScore'`	`PASS`
U1.3	Accepts valid 'date' field	`normalizeSortField('date')`	`'date'`	`PASS`
U1.4	Invalid field defaults to 'price'	`normalizeSortField('invalid')`	`'price'`	`PASS`
U1.5	Empty string defaults to 'price'	`normalizeSortField('')`	`'price'`	`PASS`

2. normalizeSortOrder Tests

Test ID	What Is Being Tested	Input	Expected Output	Result
U2.1	Accepts valid ASC direction	`normalizeSortOrder('ASC')`	`'ASC'`	`PASS`
U2.2	Accepts valid DESC direction	`normalizeSortOrder('DESC')`	`'DESC'`	`PASS`
U2.3	Invalid input defaults to ASC	`normalizeSortOrder('invalid')`	`'ASC'`	`PASS`

3. sortProperties Test

Test ID	What Is Being Tested	Input (field, order)	Expected Output	Result
U3.1	Price sorted lowest to highest	`price, ASC`	Cheapest property first, most expensive last	`PASS`
U3.2	Price sorted highest to lowest	`price, DESC`	Most expensive first, cheapest last	`PASS`
U3.3	Safety score sorted lowest to highest	`safetyScore, ASC`	Lowest score first, highest last	`PASS`
U3.4	Safety score sorted highest to lowest	`safetyScore, DESC`	Highest score first, lowest last	`PASS`
U3.5	Date sorted oldest to newest	`date, ASC`	Oldest listing first, newest last	`PASS`
U3.6	Original array not mutated by sort	`price, ASC`	Source array unchanged after sort call	`PASS`

4. filterProperties Tests

Test ID	What Is Being Tested	Filter Applied	Expected Output	Result
U4.1	No filters returns all properties	All filters empty	All properties returned	`PASS`
U4.2	Min price excludes cheaper properties	`minPrice: '1500'`	Only properties at or above £1500/month	`PASS`
U4.3	Max price excludes more expensive properties	`maxPrice: '1500'`	Only properties at or below £1500/month	`PASS`

U4.4	Price range filters both ends correctly	`minPrice: '1200', maxPrice: '1800'`	Only properties within £1200–£1800	PASS
U4.5	Min bedrooms excludes smaller properties	`minBedrooms: '3'`	Only properties with 3 or more bedrooms	PASS
U4.6	Max bedrooms excludes larger properties	`maxBedrooms: '3'`	Only properties with 3 or fewer bedrooms	PASS
U4.7	Min bathrooms filter works correctly	`minBathrooms: '2'`	Only properties with 2 or more bathrooms	PASS
U4.8	Max bathrooms filter works correctly	`maxBathrooms: '2'`	Only properties with 2 or fewer bathrooms	PASS
U4.9	House type filter: DETACHED	`houseType: 'DETACHED'`	Only detached properties returned	PASS
U4.10	House type filter: FLAT	`houseType: 'FLAT'`	Only flat properties returned	PASS
U4.11	Multiple filters applied simultaneously	`minPrice, maxPrice, minBedrooms, houseType`	Only property matching all criteria returned	PASS
U4.12	No matching results returns empty array	`minPrice: '5000'`	Empty array returned	PASS
U4.13	Boundary: property at exact minPrice is included	`minPrice: '900'`	Property priced at exactly £900 included	PASS
U4.14	Boundary: property at exact maxPrice is included	`maxPrice: '2500'`	Property priced at exactly £2500 included	PASS
U4.15	Min safety score filters low-scoring areas	`minSafetyScore: '60'`	Only properties in neighbourhoods scoring 60+	PASS

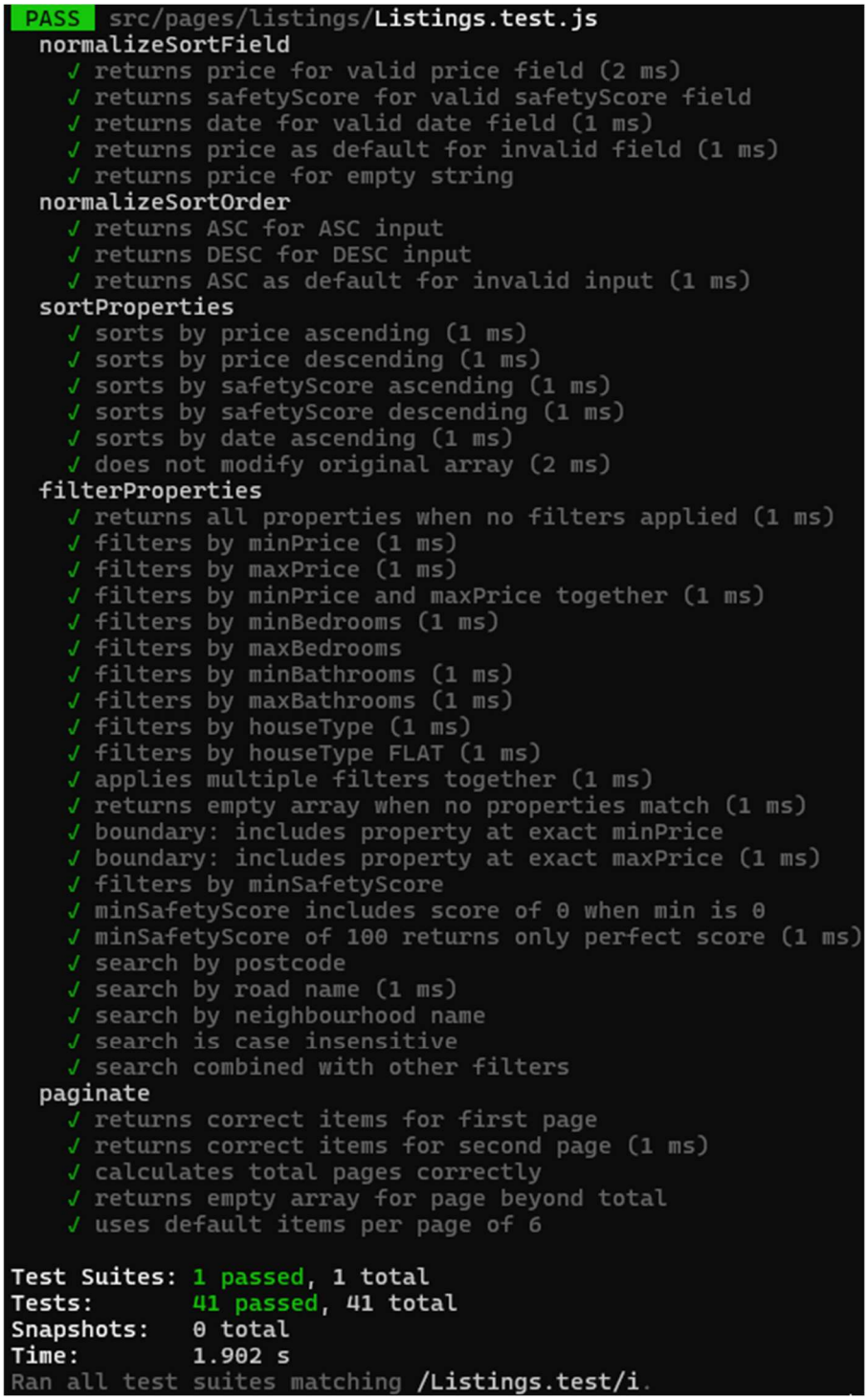
U4.16	Safety score of 0 returns all properties	`minSafetyScore: '0'`	All properties returned including score of 0	PASS
U4.17	Safety score of 100 returns only perfect score	`minSafetyScore: '100'`	Only property with score of 100 returned	PASS
U4.18	Search matches by postcode	`search: 'SW1A'`	Only properties with matching postcode shown	PASS
U4.19	Search matches by road name	`search: 'Baker'`	Only properties on matching road shown	PASS
U4.20	Search matches by neighbourhood name	`search: 'Downtown'`	Only properties in matching neighbourhood shown	PASS
U4.21	Search is case insensitive	`search: 'sw1a'` (lowercase)	Same results returned as uppercase search	PASS
U4.22	Search combined with another filter	`search + minBedrooms`	Only properties matching both criteria returned	PASS

5. Paginate Tests

Test ID	What Is Being Tested	Input	Expected Output	Result
U5.1	First page returns correct slice of results	`page=1, size=2`	First 2 properties returned	PASS
U5.2	Second page returns next slice of results	`page=2, size=2`	Properties 3 and 4 returned	PASS
U5.3	Total pages calculated correctly	`6 items, size=2`	totalPages = 3	PASS

U5.4	Page beyond total returns empty array	`page=10, size=2`	Empty array returned	PASS
U5.5	Default page size of 6 applied when not specified	`page=1, no size arg`	All items on 1 page, totalPages = 1	PASS

6. Results



Individual listing view

For the individual listing feature there is not many unit tests I can develop in code as the majority of the page's function is to visually return a property listing and enable other features (like viewing the safety score breakdown and adding listings to favourites).

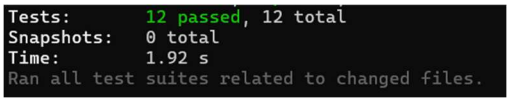
However as the page uses two different functions to format certain pages correctly:

- formatPropertyType which formats the data returned from our API for the specific property type of the listing e.g. SEMI_DETACHED to Semi Detached
- getSafetyScoreColour which returns the colour we want to display our safety score as regarding how high or low it is

these functions can be found in:

...\LogicLuminaries\safedex-frontend\src\pages\individual-listing\IndividualListing.test.js.

There are 12 tests testing each input group for both functions.



All twelve of these tests passed successfully.

For the rest of my page, the testing is best done manually. I have created this test plan iteratively as I developed the page to cover all it's requirements and expected behaviour:

Testing the overall different screens:

Test ID	Test Description	Expected Result
T1	Full screen view	The full screen view should display without any overlapping, including all the necessary property listing details and buttons.
T2	Half screen view	The half screen view should redistribute page components, so everything wraps without overlapping, shrunken or malformed components.
T3	Loading screen	The loading screen should appear whilst waiting for a response from the API before a listing is returned. It should display "Loading property..."
T4	Listing Not Found screen	Listing not found screen should be displayed if the API returns nothing, including when the backend is down or when the id in the URL does not have a corresponding listing stored. It should return Listing Not Found with a button to return to Listings search.

Testing the correct screen responses for the URL:

Test ID	Test Description	Action taken	Expected Result
T5	Valid property ID for listing currently stored in the database	URL is .../listings/4	Property details are all displayed correctly. The data should correspond to the database data for *id 4:* *ppm = 680* *neighbourhood_id = 1 (selly oak)* *bathrooms = 2* *bedrooms = 4* *description = null (auto generated)* *email = dawlish@rentals.com* *phone_number = 07100000004* *postcode = B29 7AE* *road = Dawlish Road* *type = SEMI_DETACHED* *safety_score = 48*
T6	Invalid property ID for listing not currently in database	Navigate to URL at .../listings/1000	“Listing Not Found” page should appear
T7	Invalid property ID that is not an integer	Navigate to URL at .../listings/abc	“Listing Not Found” page should appear without any errors

Testing individual components appear correctly

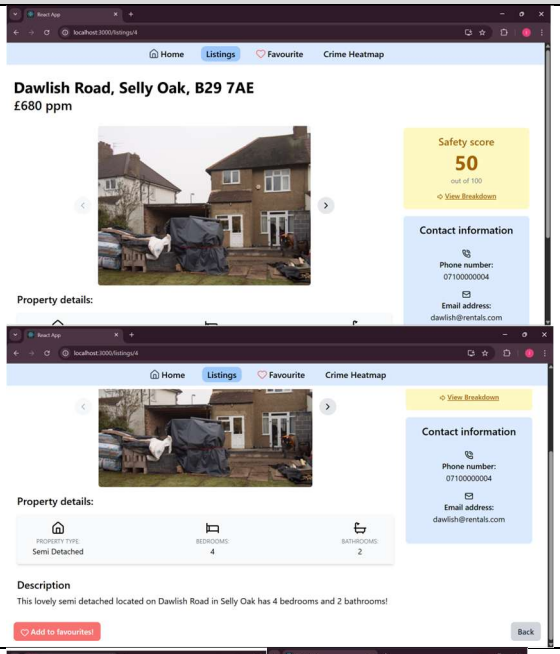
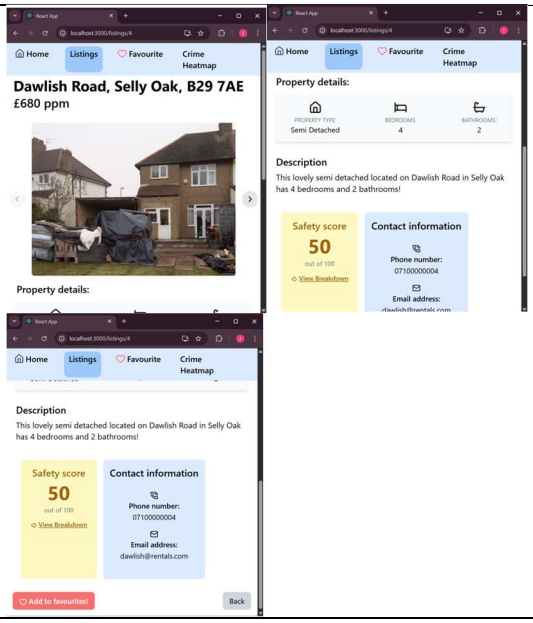
Test ID	Test Description	Action taken	Expected Result
T8	Property will description	Navigate to URL of /listings/7	Description section should display "Large family home."
T9	Property without a description (null)	Navigate to URL at /listings/1	The autogenerated description should display. In this case it should read “This lovely terraced located on Oakfield Road in Selly Oak has 3 bedrooms and 1 bathrooms!”
T10	Property with an email stored	Navigate to URL at .../listings/19	The contact info box should include an email section with the email "hurst@citylets.co.uk"
T11	Property without an email (null)	Navigate to URL at listings/22	The contact info box should just display the phone number without an email section.
T12	Property type is displayed in the correct format	Navigate to URL at listings/14	The property type should be displayed as Semi-detached in the property info row

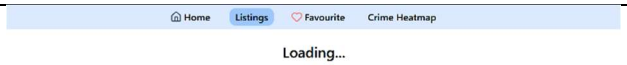
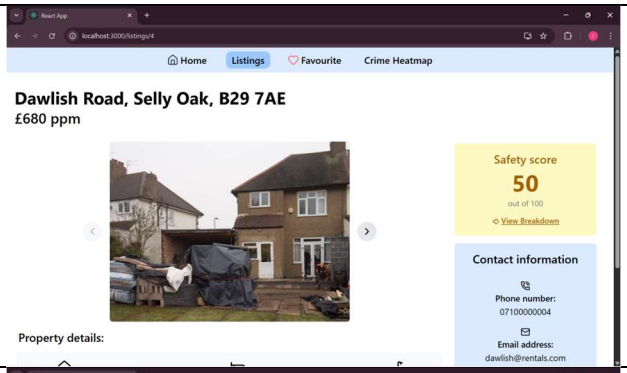
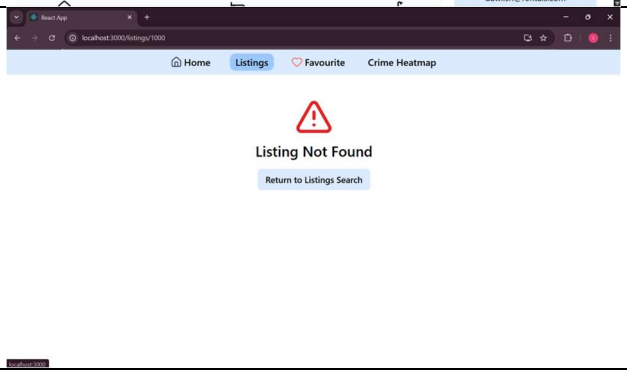
T13	Property with just one image	Navigate to URL at listings/11	The correct image should be displayed with both left and right buttons disabled
T14	Right button for property with one image	Navigate to URL at listings/11	Right button should be disabled and not do anything when clicked upon
T15	Left button for property with one image	Navigate to URL at listings/11	Left button should be disabled and not do anything when clicked upon
T16	Property with multiple images	Navigate to URL at listings/30	The correct image should be displayed at the top with left button disabled and right button enabled initially. User should be able to click through all the images.
T17	Right button for property with multiple images	Navigate to URL at listings/30 Click the image right button	The right button should change the image to the next one in the list until it reaches the end
T18	Left button for property with multiple images	Navigate to URL at listings/30 Click the image right button Click the image left button	The right button should change the image to the next one in the list until and then clicking the left button should change it back to the previous image.
T19	Back button	Navigate to URL at listings/1 Click the back button	The back button should return to the previous page. Upon integration this will usually be the listings search page but if these tests are performed in order it should be /listings/30
T20	Back to listings button on listing not found screen	Navigate to URL at listings/100 Click the back to listings button	The back to listings button should return to .../listings
T21	Add to favourites button	Navigate to URL at listings/1 Click favourites button	Upon integration this should add a listings to favourites, for now it should do nothing and not error
T22	View Safety score Breakdown button	Navigate to URL at listings/1 Click Safety score breakdown button	Upon integration this should redirect to the neighbourhood safety breakdown page. For now, since it redirects to the page /listings/neighbourhood/id which does not exist it should return to the landing page.

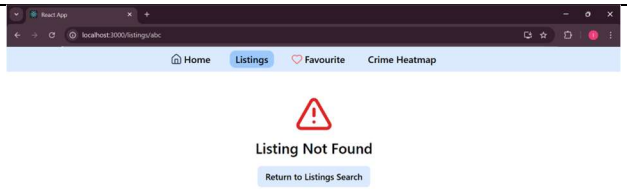
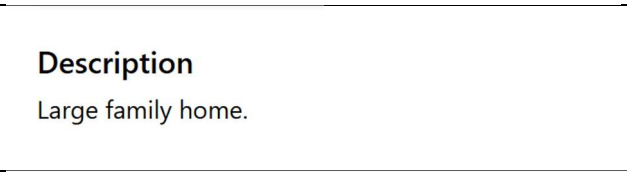
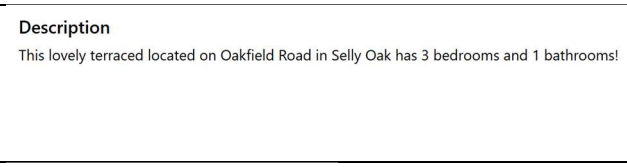
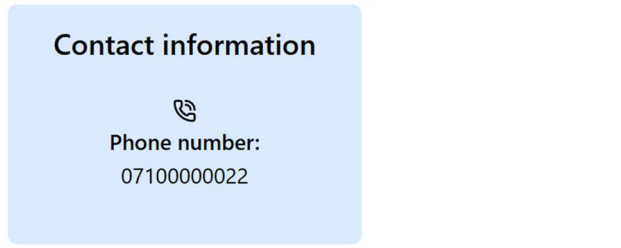
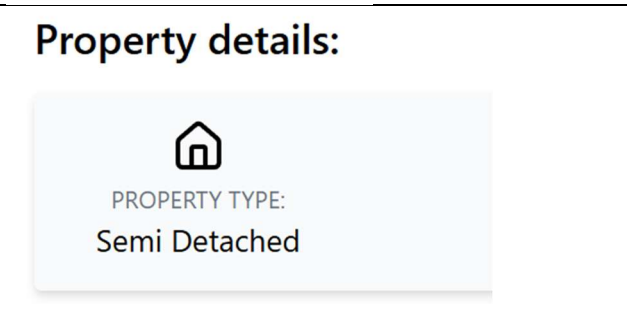
Testing keyboard accessibility:

Test ID	Test Description	Action taken	Expected Result
T23	Tabbing through buttons	Navigate to URL of /listings/1 Press tab	Upon tab you should progress through all the buttons
T24	Tabbing backwards through buttons	Navigate to URL of /listings/1 Press shift + tab	Upon progressing shift and tab it should return to the previous button on the page

Completed test plans:

Test ID	Test Description	Result	Pass?
T1	Full screen view		Yes Expected Result
T2	Half screen view		Yes Expected Result

T3	Loading screen		Yes Note most of the time the loading screen is not visible as the API responds quickly
T4	Listing Not Found screen		Yes Expected Result
T5	Valid property ID for listing currently stored in the database		Yes Expected Result
T6	Invalid property ID for listing not currently in database		Yes Expected Result

T7	Invalid property ID that is not an integer		Yes Expected Result
T8	Property with description		Yes Expected Result
T9	Property without a description (null)		Yes Expected Result
T10	Property with an email stored		Yes Expected Result
T11	Property without an email (null)		Yes Expected Result
T12	Property type is displayed in the correct format		Yes Expected Result

T13	Property with just one image		Yes Expected Result
T14	Right button for property with one image	Nothing happens when the button is pressed, button appears disabled	Yes Expected Result
T15	Left button for property with one image	Nothing happens when the button is pressed, button appears disabled	Yes Expected Result
T16	Property with multiple images		Yes Expected Result
T17	Right button for property with multiple images	 Right button switches image as expected.	Yes Expected Result
T18	Left button for property with multiple images	Left button switches image as expected.	Yes Expected Result
T19	Back button	The back button returns to the previous page as expected.	Yes Expected Result
T20	Back to listings button on listing not found screen	The back to listings button returns to .../listings as expected	Yes

			Expected Result
T21	Add to favourites button	The favourites button can be clicked and does not error but currently does nothing	Yes* Expected Result for now
T22	View Safety score Breakdown button	Currently redirects to the landing page as /listings/neighbourhood/id does not exist yet.	Yes* Expected Result for now
T23	Tabbing through buttons	You can tab through every button on the page as expected.	Yes Expected Result
T24	Tabbing backwards through buttons	You can shift tab through every button on the page as expected	Yes Expected Result

*These tests will be redone upon integration with corresponding components.

Safety score breakdown

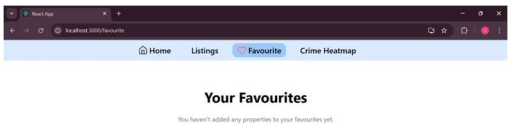
The test plan include 9 unit and 5 integration testing (with the backend) for the neighbourhood safety score breakdown page. The unit tests validate two helper functions which are getSafetyColor and getCategoryColor that are used to return the correct colours and labels on safety score and crime categories. The integration tests verify the overall behaviour of the page in the browser, which are overall data display, safety score colour, recent crimes pagination, back button navigation and error handling when backend is unavailable.

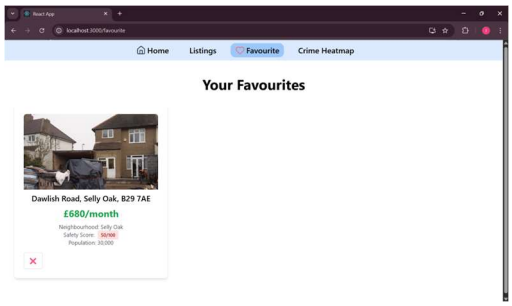
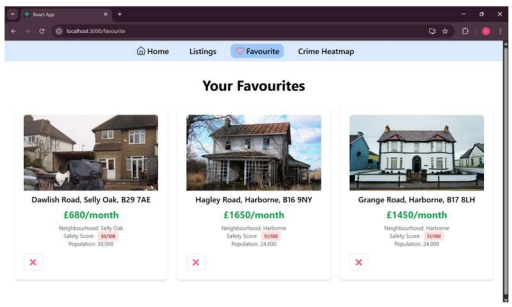
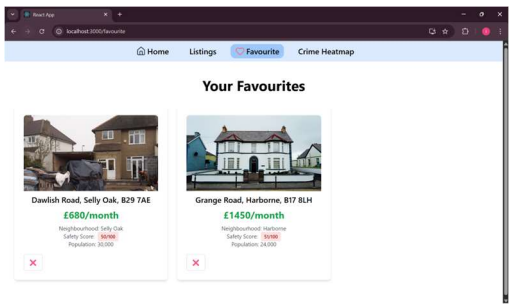
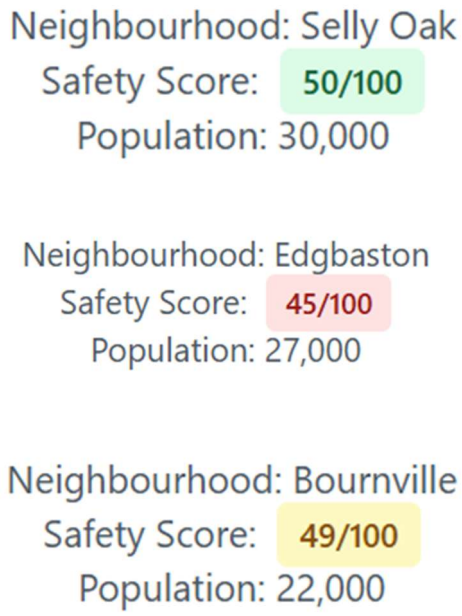
Test ID	Test description	Action taken	Expected result
T1	Safe score label	Call getSafetyColor(70)	Returns label “Safe” and green for score 70 and above
T2	Safe score boundary	Call getSafetyColor(100)	Returns label “Safe” for score 100
T3	Moderate score label	Call getSafetyColor(60)	Returns label “Moderate” and amber for score between 40 and 69
T4	Moderate score boundary	Call getSafetyColor(40)	Returns label “Moderate” for score exactly 50
T5	High Risk score label	Call getSafetyColor(30)	Returns label “High Risk” and red for score below 40
T6	High Risk score boundary	Call getSafetyColor(0)	Returns label “High Risk” for score 0
T7	Crime category colour	Call getCategoryColor('violent-crime')	Returns correct colour which is pink
T8	Crime category colour	Call getCategoryColor('anti-social-behaviour')	Returns correct colour which is blue
T9	Unknown category colour	Call getCategoryColor('unknown-category')	Returns default grey colour
T10	Safety score page loads	Navigate to URL at ../neighbourhood/1	Page displays safety score breakdown of the neighbourhood which is “Selly Oak” with its

			overall safety score card, crime breakdown bars by category and recent crimes of the latest month
T11	Moderate safety score displays amber	Navigate to URL at ../neighbourhood/3 which is a neighbourhood with score between 40 and 69	Score circle displays “45” in amber colour and "Moderate" label
T12	Pagination works	Click Next button on recent crimes at any neighbourhood safety score breakdown	Next page of crimes is displayed
T13	Back button works	Click Back button at any neighbourhood safety score breakdown	Navigates to previous page which is individual listings at ../listings/id
T14	Error handling	Turn off backend and navigate to page	Error message displays with Try Again button

Favourites page

Test	Test Description	Expected Result
Test 1	Initial Screen - no favourites added	Page displays heading “Your Favourites” and text “You haven't added any properties to your favourites yet”. No properties shown
Test 2	Add property to favourites from Listings	Heart button is clicked on property in Listings page and then navigate to Favourites. Property should show with name, price, neighbourhood info
Test 3	Add multiple properties	As above but should show multiple properties
Test 4	Remove property	Cross button clicked on property card in Favourites and property removed from Favourites
Test 5	Safety score colours	Properties will have different text colours based on their safety score out of 100

Test	Test Description	Result	Pass?
Test 1	Initial Screen - no favourites added		Yes

Test 2	Add property to favourites from Listings		Yes
Test 3	Add multiple properties		Yes
Test 4	Remove property		Yes
Test 5	Safety score colours Note all current neighbourhood safety scores are within the moderate range so in order to test we have changed the boundaries so 45-47 is red, 48 is orange, 50+ is green. This change is purely for test purposes to prove expected behaviour and has then been changed back.		Yes

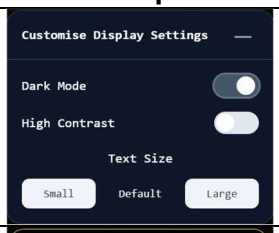
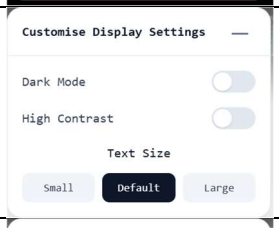
Landing page, accessibility and customisable UI

Landing page

Test	Test description	Expected response	Actual response	Pass
Test 1	Full screen layout	All components appear on screen without overlapping or any incorrect formatting	As expected	Yes
Test 2	Half screen layout	Components redistribute to prevent overlapping	As expected	Yes
Test 3	Safedex button in top left	Upon clicking, it should return back to the landing page from any other page	As expected	Yes

The rest of the landing page largely consists of hardcoded information which does not need testing and buttons routing to the other pages which will be tested upon integration.

Customisable UI and accessibility:

Test	Test description	Expected response	Actual response	Pass
Test 1	Dark mode button	Toggles dark mode		Yes
Test 2	High contrast button	Toggles high contrast mode		Yes
Test 3	Light mode	Upon turning both dark mode and high contrast mode off, light mode should be toggled		Yes
Test 4	Default text size	The default size for text is shown		Yes

Test 5	Small text size	The text and components should be shrunk to a smaller size		Yes
Test 5	Large text size	The text and components should be enlargened to a bigger size		Yes
Test 6	You cannot toggle dark and high contrast mode at the same time	If dark mode is selected and you select high contrast mode it should unselect dark mode and vice versa	As expected	Yes
Test 6	You cannot small, default or large text size at the same time	When you select one text size it should deselect what you previously selected	As expected	Yes
Test 7	Accessibility button menu	When the accessibility button is pressed the menu appears	As expected	Yes
Test 8	Close accessibility menu	When the – on the accessibility button is pressed it closes	As expected	Yes

This needs to be ensured across the entire system so the majority of tests are integration tests with other components of the system.

Integration testing across the frontend components

This is easiest divided across pages and how they interact, primarily through buttons. The most reasonable way to format our frontend integration tests seems to be by performing the testing for each page checking each component that may affect another page. After that we will perform the customisable UI integration testing and Accessibility testing for every page.

Landing Page

Test ID	Test Description	Expected Result	Actual Result	Pass?
T1	Search property listings button at top of page	Redirects to listing search page	As expected	Pass
T2	Home button at the bottom of page	Does nothing as already on home page	As expected	Pass
T3	Listings button at the bottom of page	Redirects to listing search page	As expected	Pass
T4	Favourites button at the bottom of page	Redirects to favourites page	As expected	Pass
T5	Map button at the bottom of page	Redirects to crime map page	As expected	Pass

Listings search page

Test ID	Test Description	Expected Result	Actual Result	Pass?
T1	View an individual listing	Upon selecting the view details button for a listing, the corresponding individual listings page should display	As expected	Pass
T2	Favourite a listing	Upon clicking a grey heart it should turn red to indicate it is favourited.	As expected Note: you can only favourite a listing, not unfavourite one	Pass

Individual listing page

Test ID	Test Description	Expected Result	Actual Result	Pass?
T1	Viewing safety score breakdown	Upon clicking the view breakdown button, it should take you to the corresponding	As expected	Pass

		neighbourhood safety score breakdown page		
T2	Favourite a listing	Upon clicking Add to favourites! button it changes to Added to favourites and that listing should now appear in the favourites page	As expected Note: you can only favourite a listing, not unfavourite one	Pass
T3	Back button	Back button should return you to the previous page (usually the listing search unless accessed directly by URL)	As expected	Pass

Neighbourhood safety score breakdown page

Test ID	Test Description	Expected Result	Actual Result	Pass?
T1	Back button	Back button should return you to the previous page (usually the individual listing page unless accessed directly by URL)	As expected	Pass

Favourites page

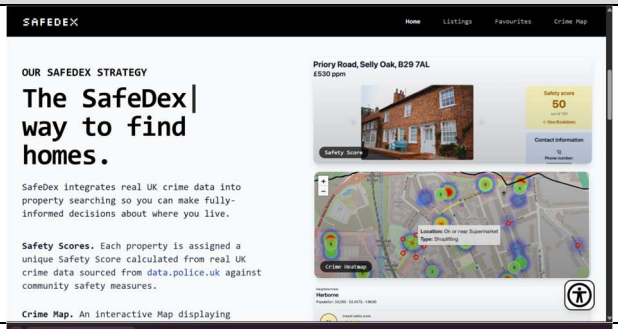
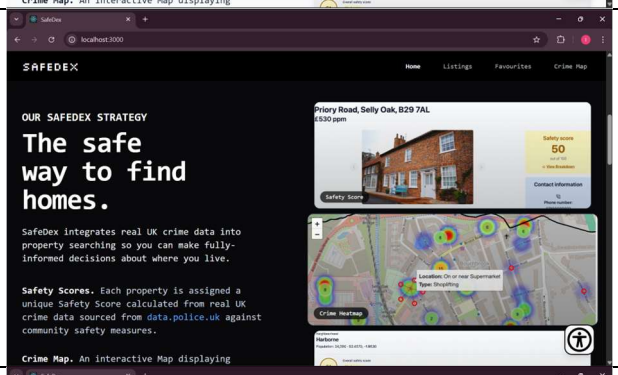
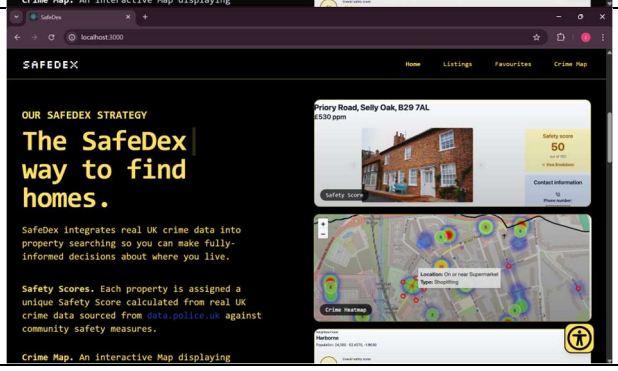
Test ID	Test Description	Expected Result	Actual Result	Pass?
T1	Favourited listings shown	When the favourite button is pressed on either listing search or individual listing page, that listing should appear in favourites	As expected	Pass
T2	Remove a listing	When the X is pressed the listing is removed, now in listings search that property should have a grey heart not a red one	As expected	Pass

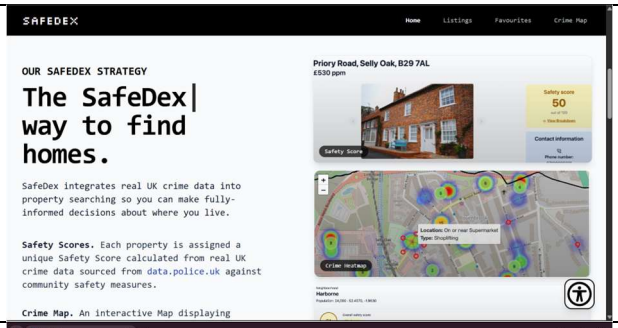
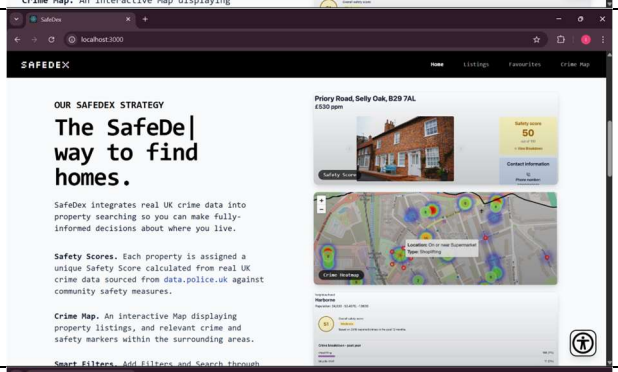
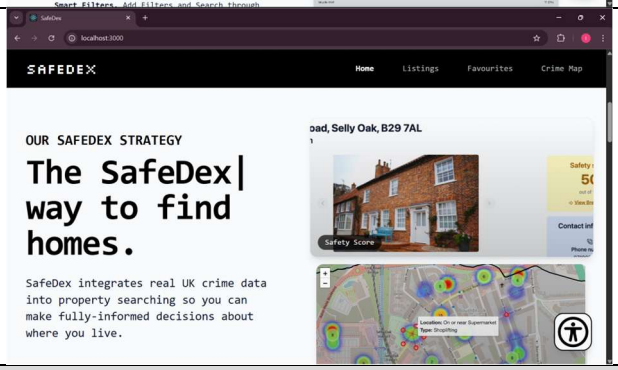
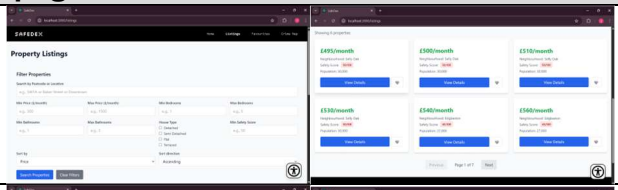
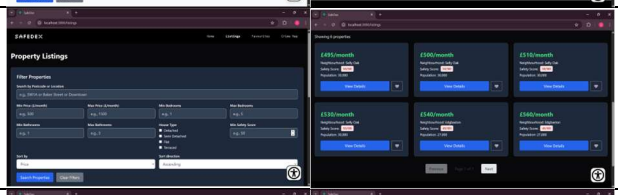
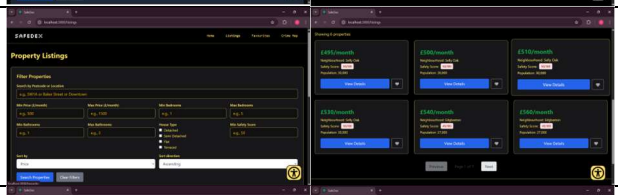
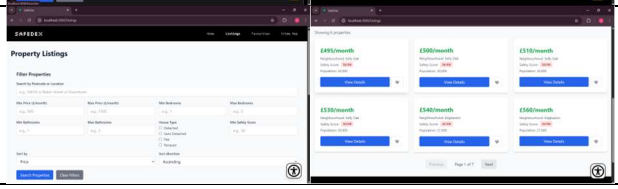
Customisable UI

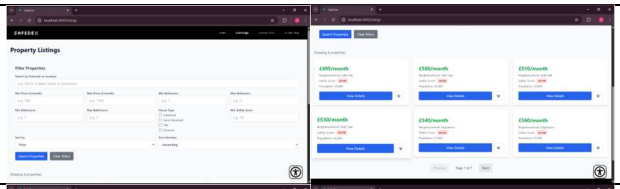
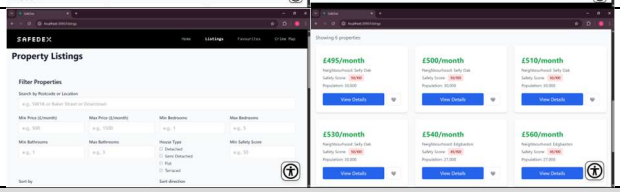
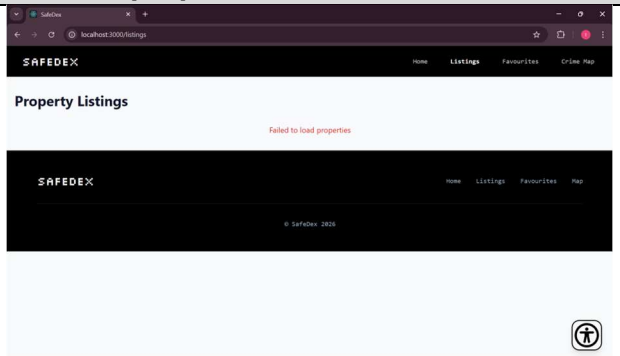
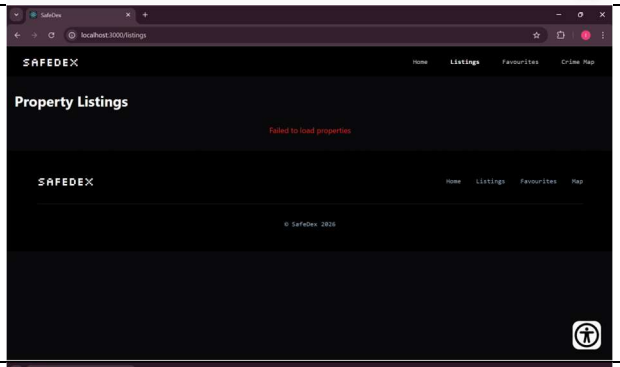
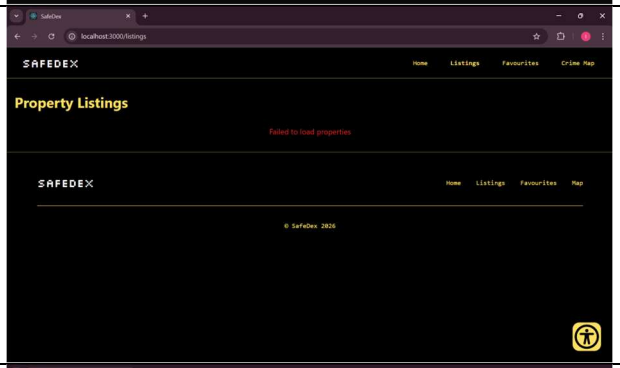
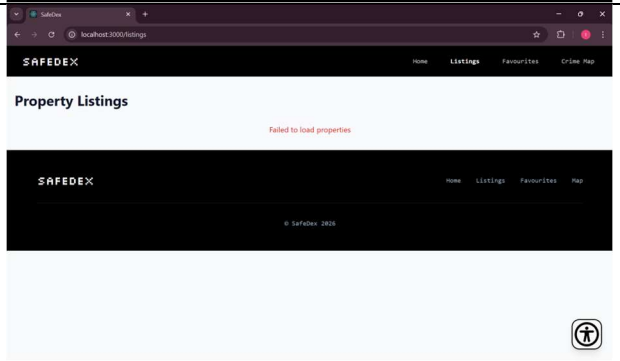
For every page we have the following tests:

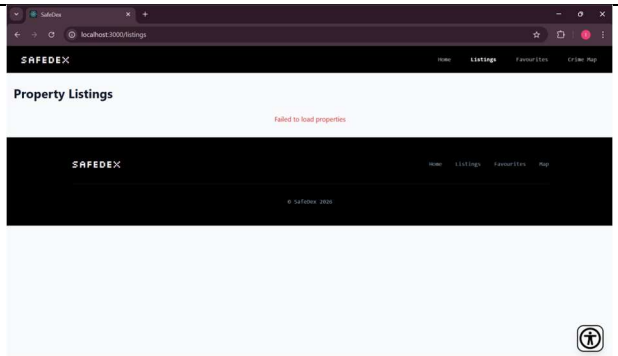
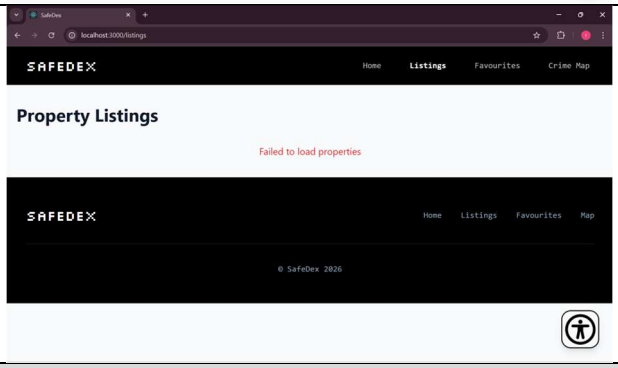
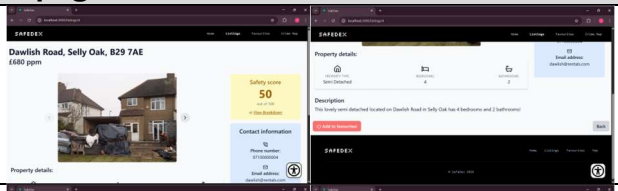
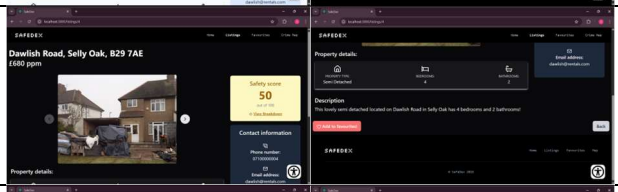
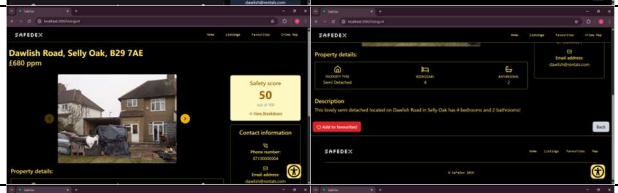
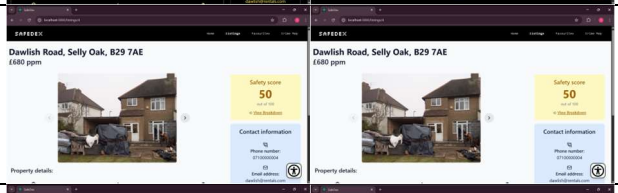
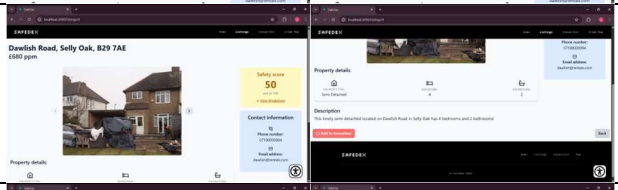
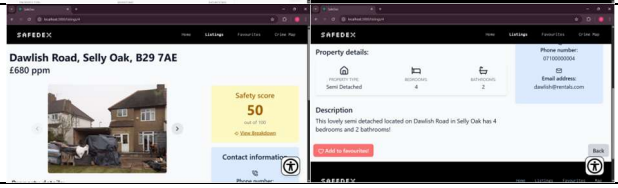
Test ID	Test Description	Expected Result
T1	Light mode	When neither dark or high contrast mode is enabled, the UI appears in light mode

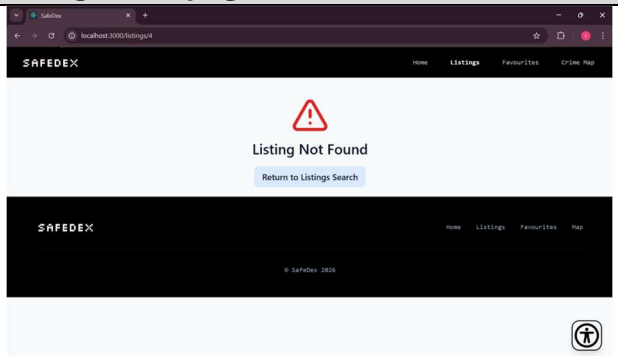
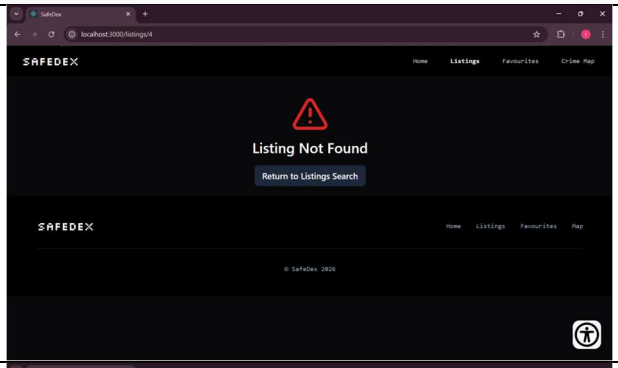
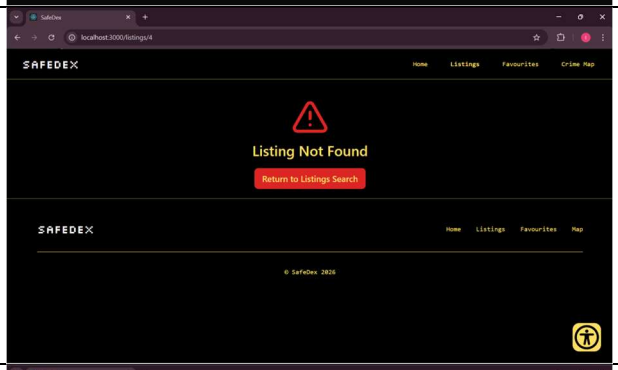
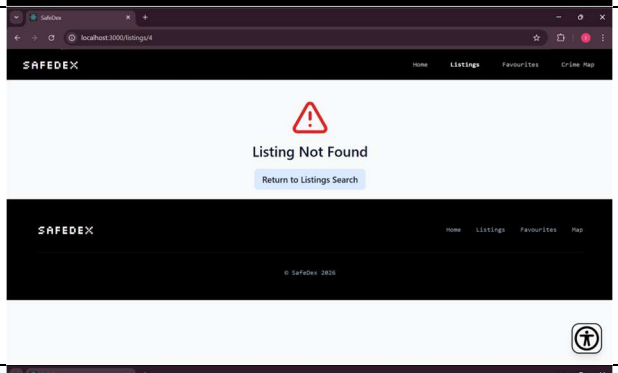
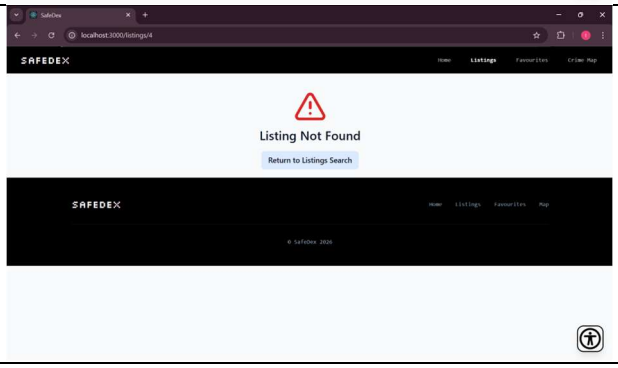
T2	Dark mode	When the dark mode button is toggled, the UI appears in dark mode
T3	High contrast mode	When the high contrast mode is toggled the UI appear in high contrast mode
T4	Default text size	When the default text size is selected, the UI appears in normal size
T5	Small text size	When the small text size is selected, the UI appears in a smaller size
T6	large text size	When the large text size is selected, the UI appears in a larger size

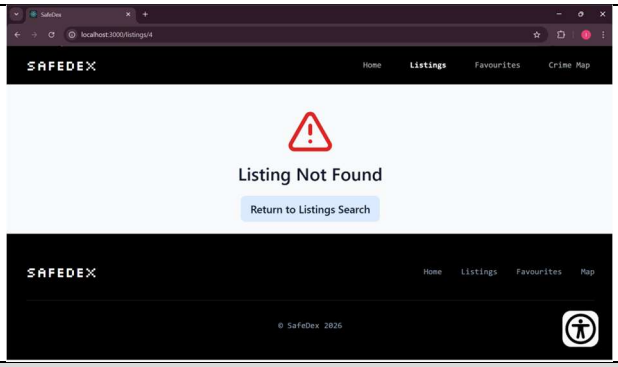
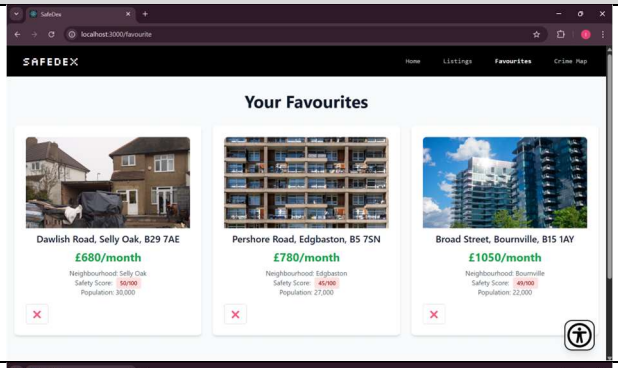
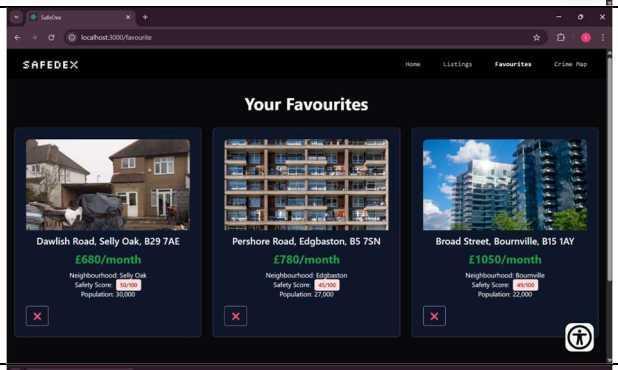
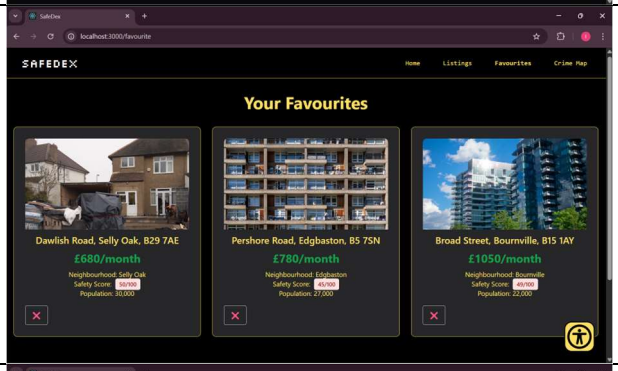
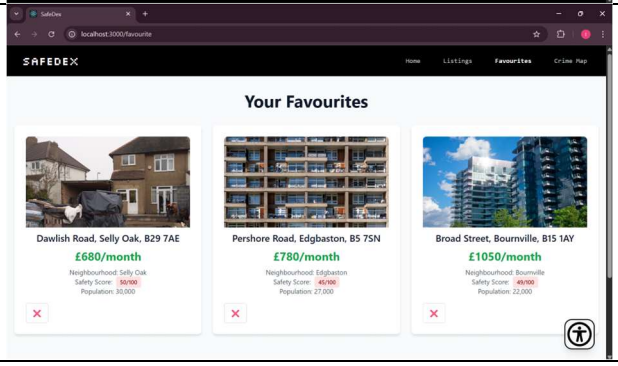
Test	Description	Result	Pass
Landing page			
T1	Light mode		Yes
T2	Dark mode		Yes
T3	High contrast mode		Yes

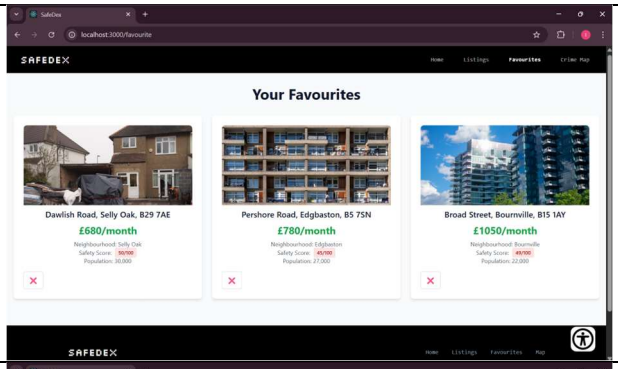
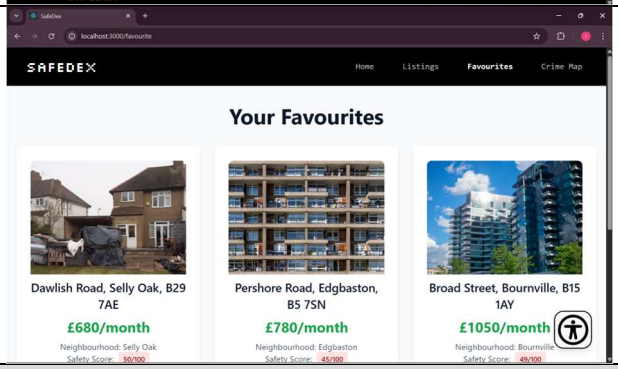
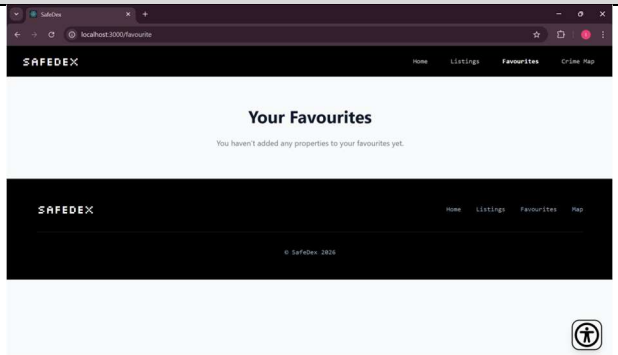
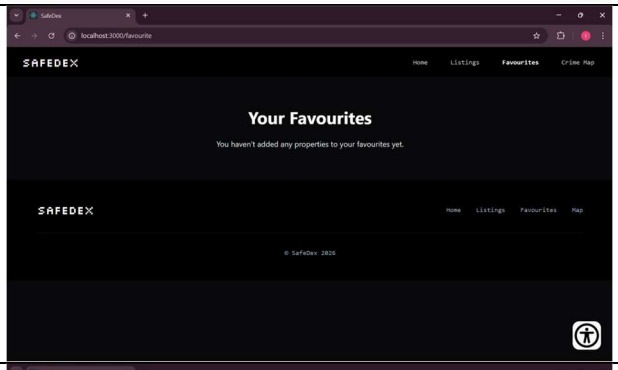
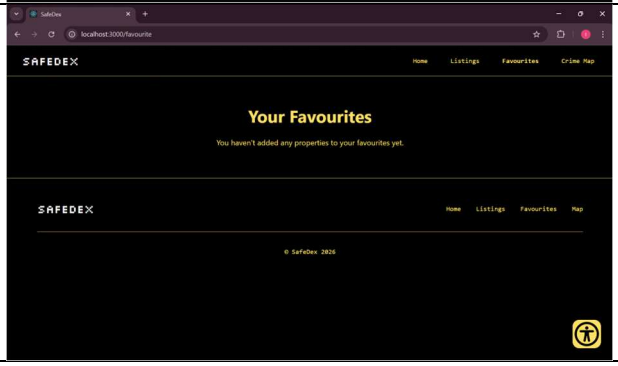
T4	Default text size			Yes
T5	Small text size			Yes
T6	large text size			Yes
Listings search page – main page				
T1	Light mode			Yes
T2	Dark mode			Yes
T3	High contrast mode			Yes
T4	Default text size			Yes

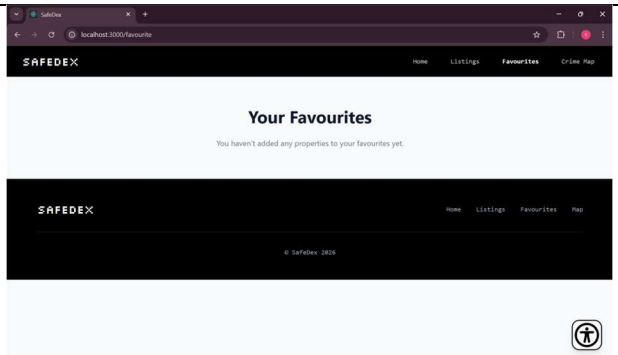
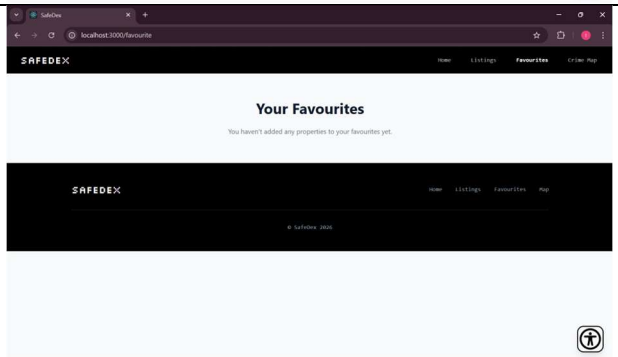
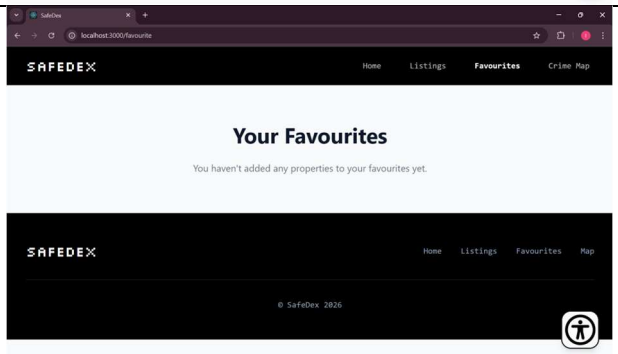
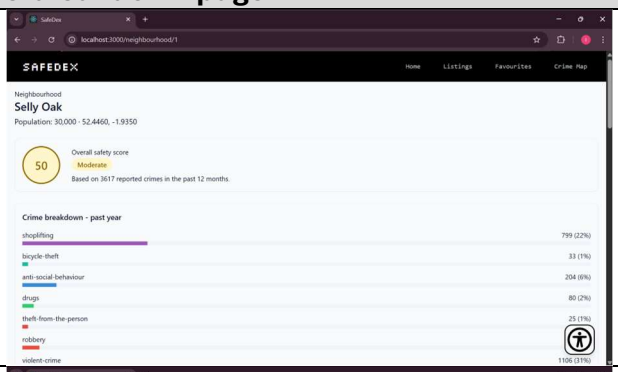
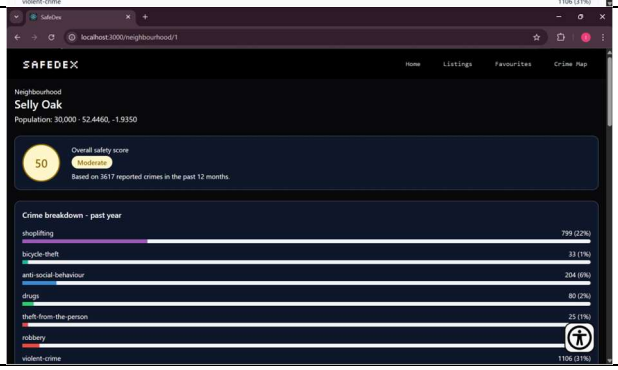
T5	Small text size		Yes
T6	large text size		Yes
Listings search page – failed to load properties			
T1	Light mode		Yes
T2	Dark mode		Yes
T3	High contrast mode		Yes
T4	Default text size		Yes

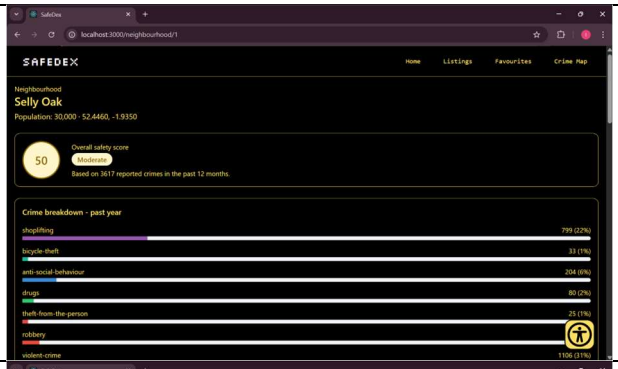
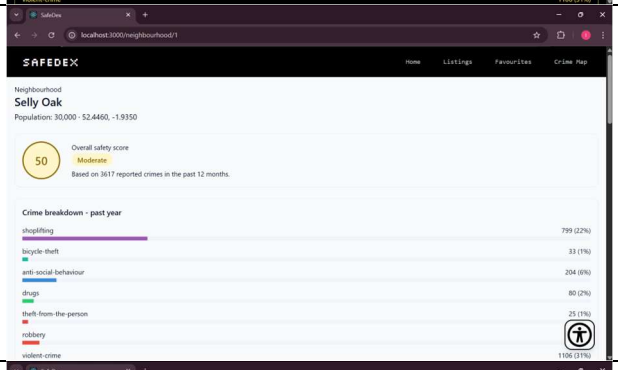
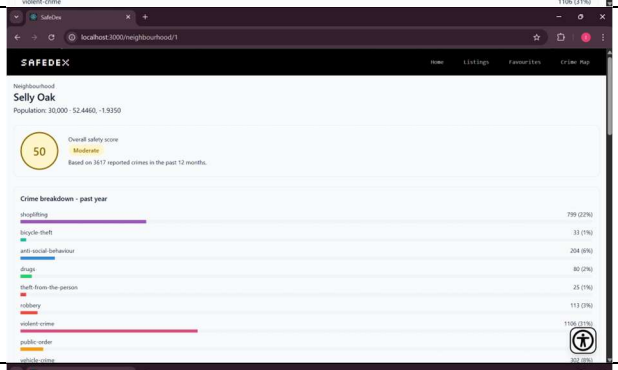
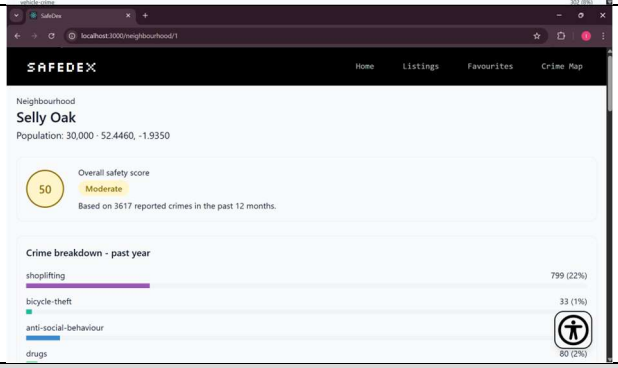
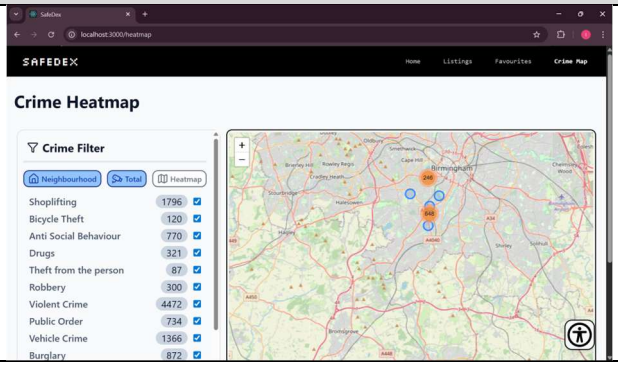
T5	Small text size			Yes
T6	large text size			Yes
Individual listing page – main page				
T1	Light mode			Yes
T2	Dark mode			Yes
T3	High contrast mode			Yes
T4	Default text size			Yes
T5	Small text size			Yes
T6	large text size			Yes

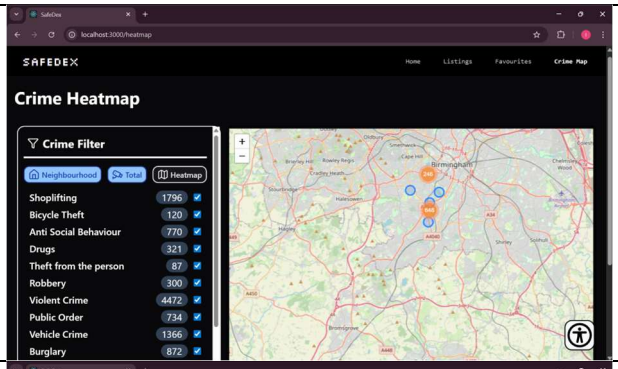
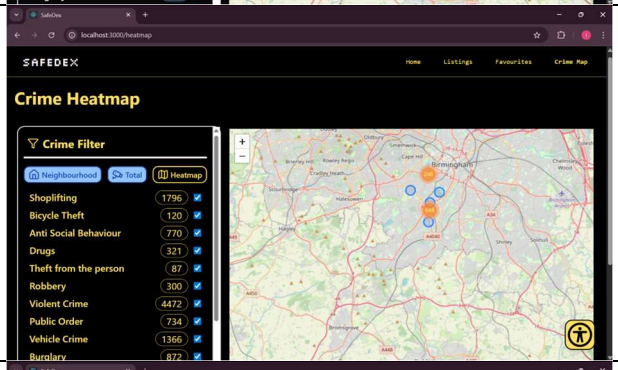
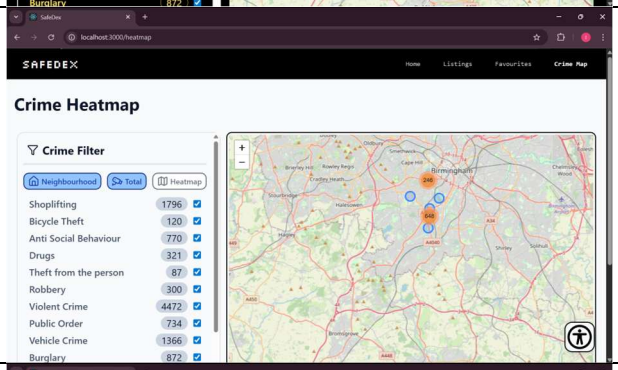
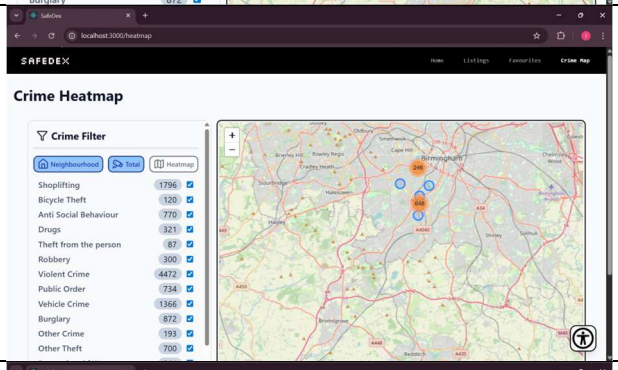
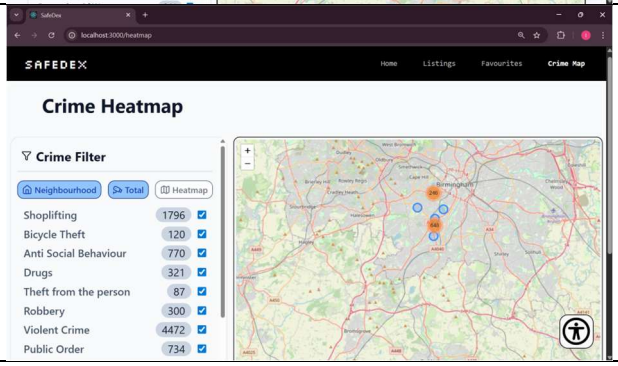
Individual listing page – no listing found page			
T1	Light mode		Yes
T2	Dark mode		Yes
T3	High contrast mode		Yes
T4	Default text size		Yes
T5	Small text size		Yes

T6	large text size			Yes
favourites page – main				
T1	Light mode			Yes
T2	Dark mode			Yes
T3	High contrast mode			Yes
T4	Default text size			Yes

T5	Small text size			Yes
T6	large text size			Yes
Favourites page - empty				
T1	Light mode			Yes
T2	Dark mode			Yes
T3	High contrast mode			Yes

T4	Default text size		Yes
T5	Small text size		Yes
T6	large text size		Yes
Neighbourhood safety score breakdown page			
T1	Light mode		Yes
T2	Dark mode		Yes

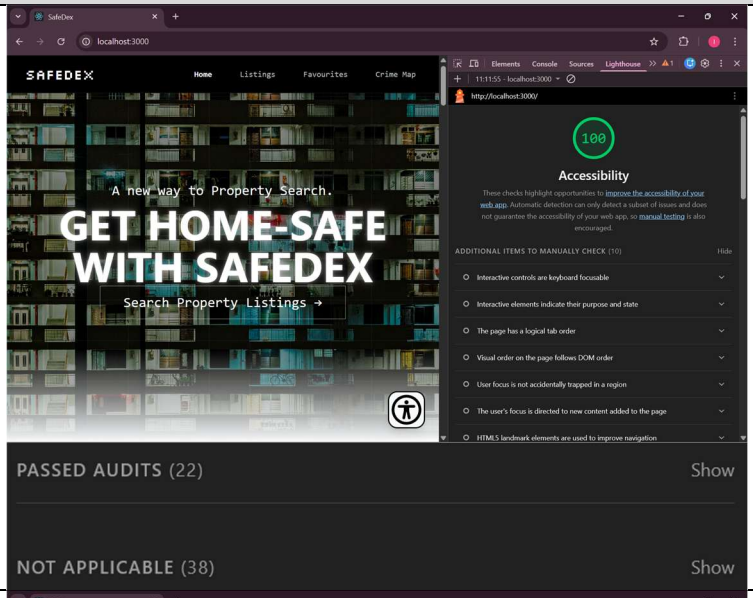
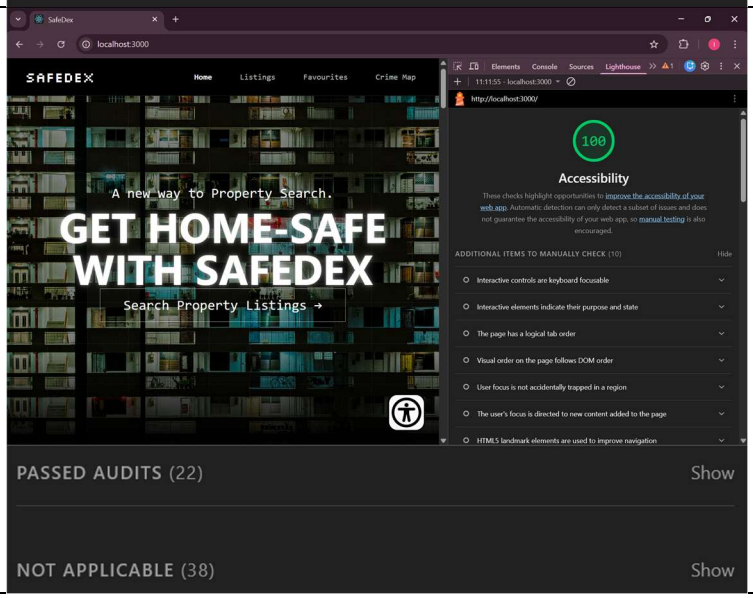
T3	High contrast mode			Yes
T4	Default text size			Yes
T5	Small text size			Yes
T6	large text size			Yes
Crime map page				
T1	Light mode			Yes

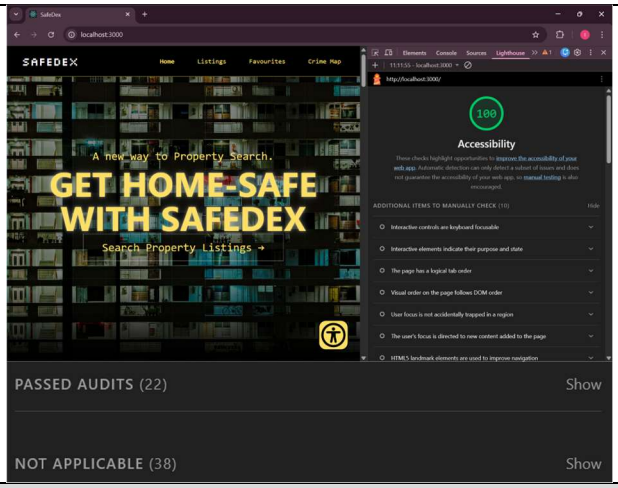
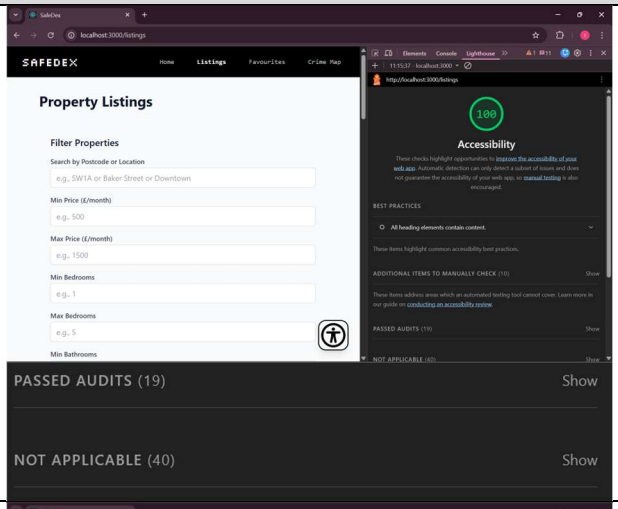
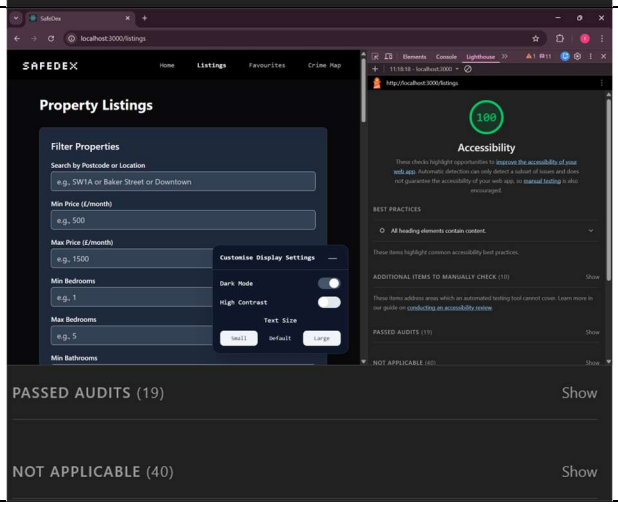
T2	Dark mode			Yes
T3	High contrast mode			Yes
T4	Default text size			Yes
T5	Small text size			Yes
T6	large text size			Yes

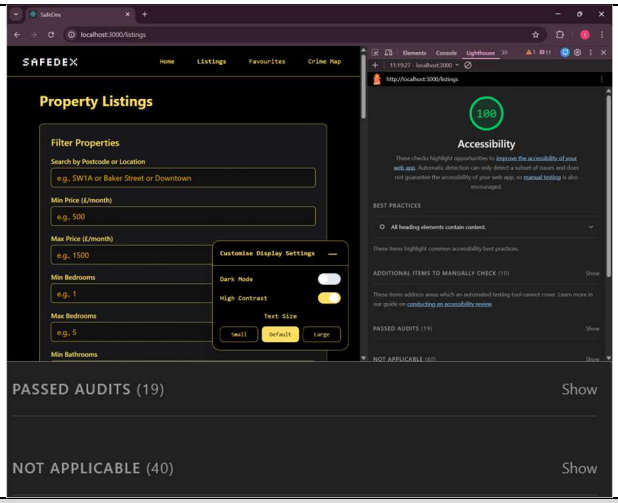
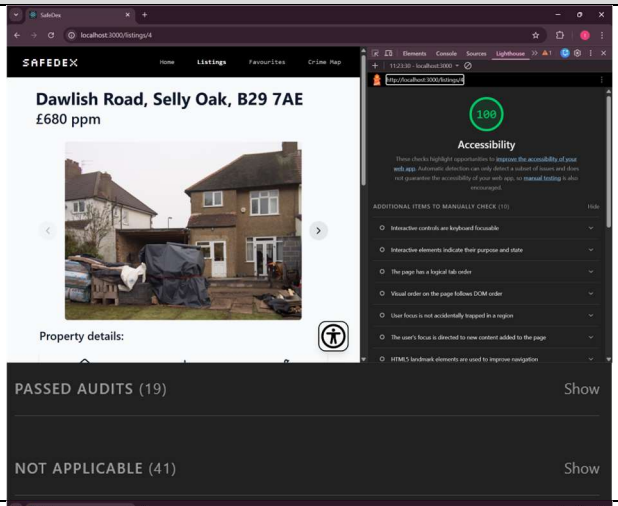
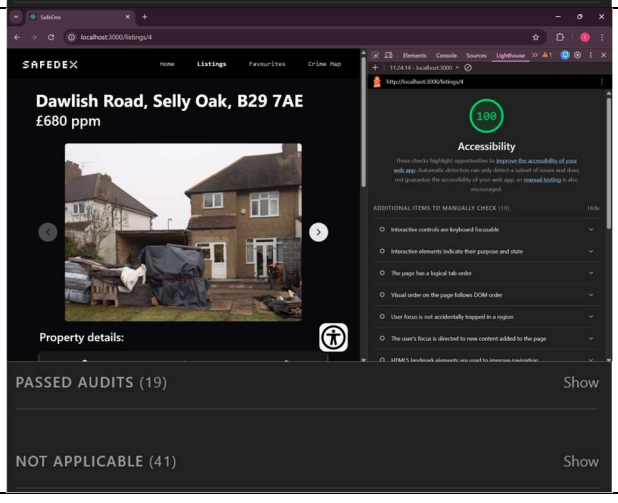
Accessibility

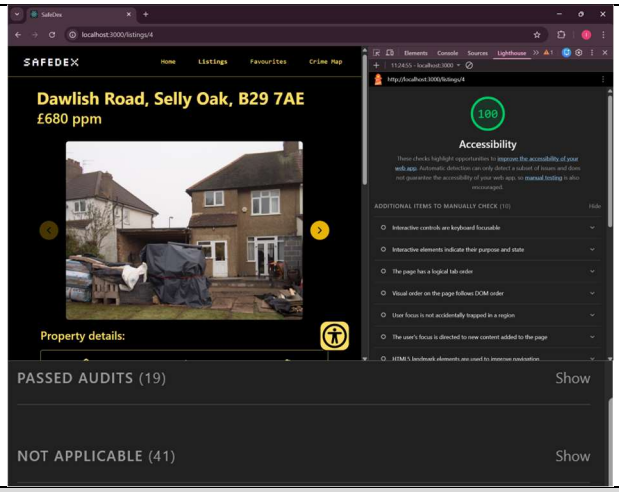
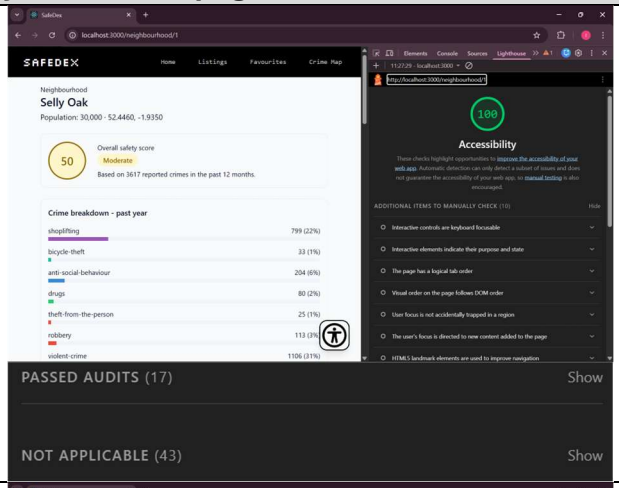
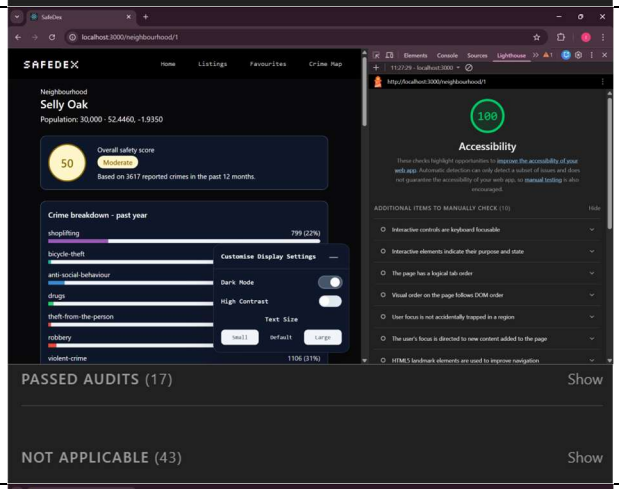
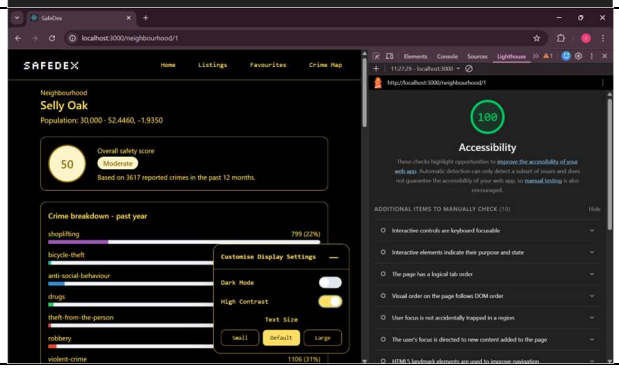
Accessibility considerations were one of our main focuses across our whole project. We had multiple features ensuring there were a variety of accessibility components across the whole prototype and it has been at the forefront of our planning, designing and development. There are some tests considering accessibility in various previous sections, for example checking the customisable UI works for every page, checking each component of the page was keyboard accessible, ensuring pages could be resized correctly, etc.

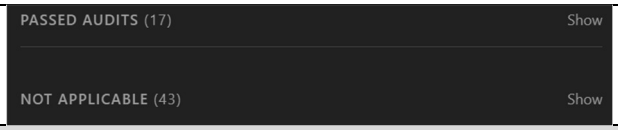
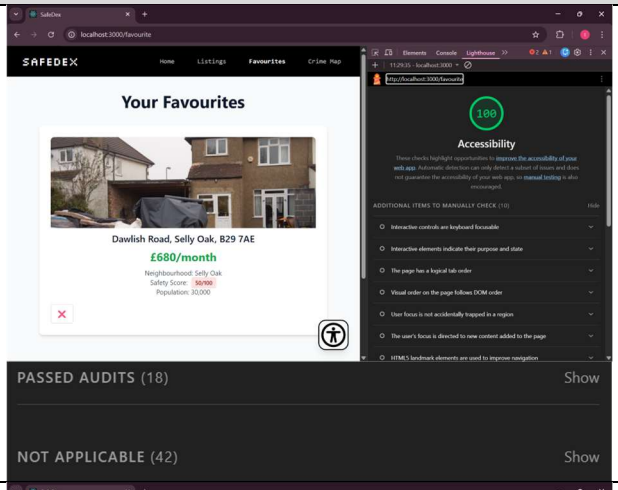
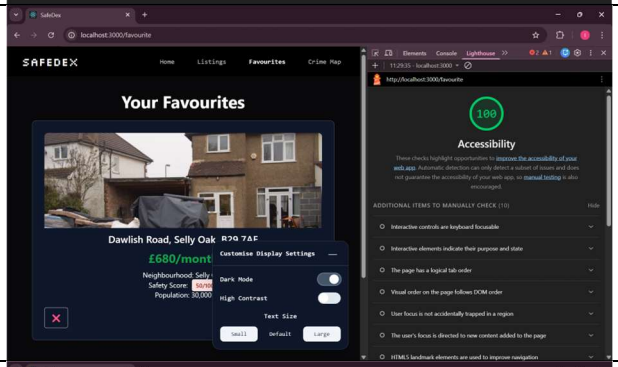
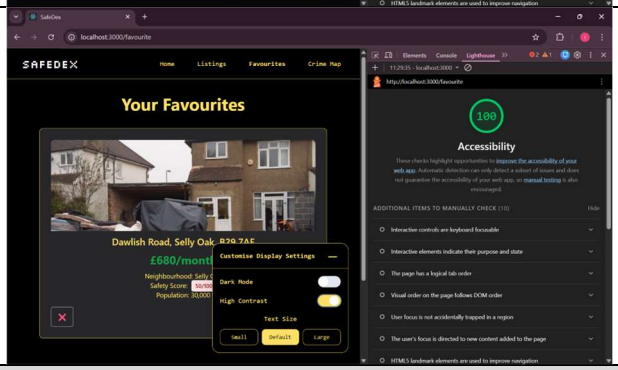
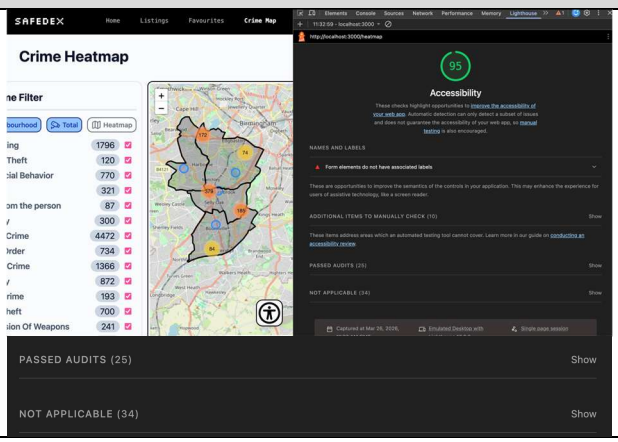
To test overall accessibility we have run the lighthouse tool measuring overall accessibility for every page – including the dark and high contrast modes.

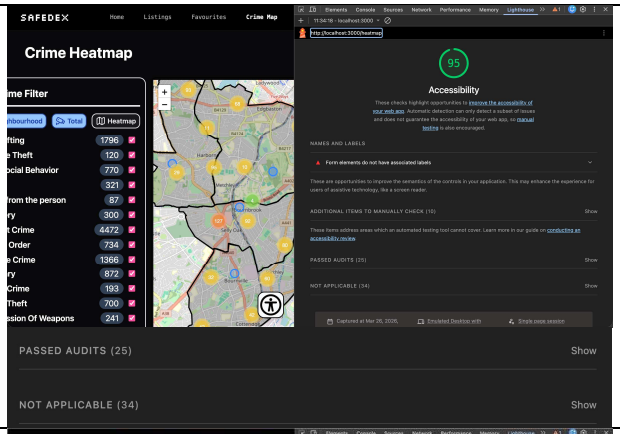
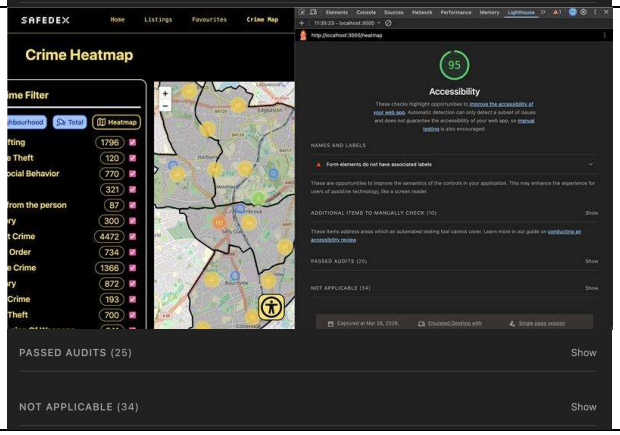
Description	Result	Pass
Landing page		
Light mode		Yes
Dark mode		Yes

High contrast mode			Yes
Listing Search page			
Light mode			Yes
Dark mode			Yes

High contrast mode			Yes
Individual Listing Page			
Light mode			Yes
Dark mode			Yes

High contrast mode			Yes
Neighbourhood Safety Breakdown page			
Light mode			Yes
Dark mode			Yes
High contrast mode			Yes

			
Favourites page			
Light mode			Yes
Dark mode			Yes
High contrast mode			Yes
Map page			
Light mode			Yes

Dark mode			Yes
High contrast mode			Yes

The Accessibility is 100 for all the pages (excluding map which only has 95 as the checkboxes do not have names but we will still consider that a pass for our first prototype).

Testing Conclusion

Overall, our testing was extremely successful with almost all tests passing with rare exceptions in which the results of the tests still behaved appropriately, even if they were not optimal (for example, the favourites button not being toggleable).

To improve upon our testing for a hypothetical second prototype of Safedex, we would perhaps encourage a more uniform standard for the group to follow when testing. Of course, different features and components require different types and styles of tests. However, it may have been helpful if we decided on a plan across the group for how we conduct our testing so each component was tested thoroughly to the same standard. Besides that, we would consider our testing to be an overall success.

Our testing demonstrates that our first prototype of Safedex is largely successful as the product achieves all the features we decided to develop in Milestone 3 as well as some of the features from Milestone 2 that were originally cut from development. The product meets user requirements and produces a robust, accessible, cohesive and complete first prototype.